markpdf Showcase

A comprehensive demonstration of every feature in markpdf. This document was generated from a single markdown file.

Why This Exists

> *We needed a tool that turns markdown into **gorgeous PDFs** -- not the bland, unstyled output you get from most converters. Something with real typography, color, and visual hierarchy.*

markpdf parses markdown into blocks, resolves images concurrently, and renders everything through reportlab with carefully tuned styles.

Text Formatting

Markdown supports a rich set of inline formatting options:

- **Bold text** for strong emphasis
- *Italic text* for subtle emphasis
- ***Bold italic*** when you really mean it
- `inline code` for technical terms and commands
- ~~Strikethrough~~ for corrections
- Highlighted text for calling attention
- Hyperlinks that are clickable in the PDF
- Superscript for math: $E = mc^2$
- Subscript for chemistry: H_2O, CO_2

You can combine these freely: **bold with** `code` **inside**, *italic with a link*, or even highlighted **bold** text.

Images

The converter supports local and remote images. Remote images are fetched **concurrently** using `aiohttp`:

Aerial view of ocean waves

Code Blocks

Code blocks get a clean, bordered container with a language label:

python

```python
import asyncio
import uvloop
from pathlib import Path

async def convert(input_path: str, output: str) -> None:
    """Convert markdown to a beautiful PDF."""
    content = await aiofiles.open(input_path).read()
    blocks = parse_markdown(content)
    remote_cache = await prefetch_images(blocks)
    story = build_story(blocks, styles, remote_cache)
    doc.build(story)
    print(f"Created: {output}")

if __name__ == "__main__":
    uvloop.install()
    asyncio.run(convert("README.md", "readme.pdf"))
```

rust

```rust
fn main() -> Result<(), Box<dyn std::error::Error>> {
    let markdown = std::fs::read_to_string("input.md")?;
    let config = ConfigSource::File("style.toml");
    parse_into_file(markdown, "output.pdf", config, None)?;
    println!("Done!");
    Ok(())
}
```

sql

```sql
SELECT u.name, COUNT(d.id) AS documents,
       AVG(d.page_count) AS avg_pages
FROM users u
JOIN documents d ON d.author_id = u.id
WHERE d.created_at > '2026-01-01'
GROUP BY u.name
HAVING COUNT(d.id) > 5
ORDER BY documents DESC;
```

Tables

Comparison Table

Feature	Our Converter	markdown2pdf (Rust)	Puppeteer

Language	Python	Rust	Node.js
Async I/O	**Yes**	No	No
Remote images	**Concurrent**	Sequential	Via Chrome
Custom styling	**Full**	TOML config	CSS
Image support	**Local + Remote**	Local + Remote	Full HTML
Code labels	**Yes**	No	Via CSS
Task lists	**Yes**	No	Yes
Highlight/Sub/Sup	**Yes**	No	Via HTML

Project Status

Phase	Description	Status	ETA
v1.0	Basic markdown parsing	**Complete**	Done
v2.0	Tables, code blocks, images	**Complete**	Done
v3.0	Async, uvloop, beautiful styling	**Complete**	Done
v3.1	Highlight, superscript, subscript	**Complete**	Done
v4.0	Nested lists, footnotes	*Planned*	Q2 2026

Lists

Bullet List

- Async file I/O with `aiofiles` for non-blocking reads
- Concurrent remote image fetching via `aiohttp`

- uvloop for **2-4x faster** event loop performance
- Pillow for reliable image loading and proportional scaling
- reportlab for pixel-perfect PDF rendering

Numbered List

1. Parse the markdown into a flat list of typed blocks
2. Scan for remote image URLs and fetch them all concurrently
3. Create reportlab styles with carefully tuned typography
4. Build the story (list of flowables) from the parsed blocks
5. Render the PDF with page numbers and footer line

Task List

- ■ Headers H1-H6 with underlines on H1/H2
- ■ Bold, italic, bold-italic, strikethrough
- ■ Inline code with pink text on gray background
- ■ Fenced code blocks with language labels
- ■ Tables with dark headers and alternating stripes
- ■ Blockquotes with blue accent border
- ■ Bullet, numbered, and task lists
- ■ Local and remote images with captions
- ■ Horizontal rules as centered decorative dividers
- ■ YAML frontmatter stripping
- ■ Highlight, superscript, subscript
- ■ Page numbers with footer separator line
- ■ Nested lists (sub-items)
- ■ Footnotes
- ■ Table of contents generation

Blockquotes

> ***Design is not just what it looks like and feels like. Design is how it works.*** *-- Steve Jobs*

> *The converter handles multi-line blockquotes gracefully. Each line starting with `>` is collected into a single block, and the whole thing gets a distinctive blue accent border with a light background.*

> *Technical note: blockquotes use a reportlab `Table` with a 4px-wide colored column on the left and the quote text on the right. This gives us the accent-border effect that CSS `border-left` provides on the web.*

Architecture

The converter follows a clean three-phase pipeline:

1. **Parse** -- Markdown text is tokenized into a list of `(block_type, content)` tuples. The parser handles frontmatter stripping, code fence detection, table parsing, and all inline formatting via regex.
1. **Resolve** -- Any remote image URLs found in the parsed blocks are fetched concurrently using `aiohttp` . Local images are resolved relative to the markdown file path.
1. **Render** -- Each block is converted to a reportlab flowable (Paragraph, Table, ListFlowable, Image, etc.) and appended to the story. The story is then built into a PDF with page numbers.

```
■■■■■■■■■■■■■     ■■■■■■■■■■■■■■     ■■■■■■■■■■■■■
■ Markdown ■ --> ■  Blocks   ■ --> ■   PDF    ■
■   Text   ■     ■ (parsed)  ■     ■ (styled) ■
■■■■■■■■■■■■■     ■■■■■■■■■■■■■■     ■■■■■■■■■■■■■
     ■                ■                  ■
  frontmatter     async image        reportlab
  stripping       prefetch           rendering
```

Generated by markpdf.